

ActInf ModelStream 009.1 ~ Aswin Paul "On efficient computation in active inference"

Source: [YouTube](#) Playlist: ModelStreams Duration: 1:00:12 | Uploaded:

Unknown Transcript method: auto_caption

foreign it's July 15 2023 we're here in active inference model stream number 9.1 with Aspen Paul today we're going to have a presentation and a discussion on efficient computation in active inference so if you're watching along live please feel free to add comments or questions in the chat otherwise thanks so much for joining today really looking forward to your talk thank you Daniel thank you so much so as mentioned today I'm going to talk about efficient computation and active inference and yeah let's get started so we're all familiar with this idea of the free energy principle which is also known as active inference right so the central concept is that an agent minimizes entropy of its observation to maintain homeostasis or survive in its environment and here the entropy is defined in the information theoretic sense right so if an observation is highly probabilistic that is less entropic or less surprising because it was high probability and we were expecting it and and and that that's the idea when that's the base that we build this framework of active inference on and this idea of Marco blanket uh it gives us a systematic way of surprising uh or the schematic way of separating uh an agent from its environment and model purposeful Behavior right so let's focus on this idea of minimizing entropy so how does an agent minimize entropy or know which observation is highly probabilistic and vice versa so that is by maintaining a generative model and the generative model is basically a toy model of the environment which the agent builds in its brain and that is built using only the observation that it gets from the environment so it has no access to the real uh States or the hidden states of the environment it's building the toy model and given this toy model it has scope or ability uh to compute the probability of an observation and hence try to minimize the entropy right so that's the idea but it has a problem of um cursive dimensionality like given a generative model it may not be possible always to calculate or marginalize the probability of observations out of it because the state space can quickly become intractable but the idea is that you define an upper bound on the surprise using Jensen's inequality and you may also Define a new term called Q which is the hidden belief or

the belief about the hidden States and this queue is going to be the focus of decision making right so if you have a noisy queue and you have no idea what's in the environment then you can't make or hope to make decisions to control that environment and it is this belief about the hidden states that you use and that becomes useful to take decisions and this whole quantity is of course called the free energy and the variational free energy F can be interpreted in multiple ways so the first or the most common is the machine learning way of how it is um Trying to minimize the complexity of the model at the same time trying to maximize the accuracy of it so that's the machine learning interpretation of minimizing variational free energy you may also try to um interpret free energy in the physics term where you at the same time try to minimize the energy of your model but at the same time trying to maximize the entropy but the focus of today or decision making is always on this belief that you have after you do the perception in active inference so how do you do vanilla decision making or what is the most discussed idea of decision making and classical active inference uh so if you are in an environment uh and if you're an agent who's trying to make decisions then you have a space of available actions right so in this toy model you have three available actions uh run jump or stay and given these actions you can hope to define a policy by π which is a sequence of actions in time and capital t is the time Horizon of planning or that's the length of your policy and you have a policy space with many such policies π so given this bigger smallest space you can attempt to evaluate the expected free energy based on the beliefs that you have accumulated so here you're not minimizing anything you already have a belief from variation for energy and you you are just calculating or evaluating the expected free energy corresponding to many uh small policies that you can Define and after you evaluate it for all policies then you know which is the optimal policy to take and that's the classical active inference idea of decision making right and this expected free energy is really useful in the sense that it is goal directed so the risk term is goal directed and then you have this expected ambiguity term that forces you to explore as well but there is a problem that this policy space can quickly turn intractable and that has always stayed as a bottleneck in scaling active inference to um commonly seen environments but let's see how it becomes quickly intractable so how many policies can be defined say for a Time Horizon of 15 right so if you are playing say Super Mario uh you might want to plan at least say 10 time step ahead so the first policy could be um the same action run um stacked for 15 time steps and you can basically Define several combinations of such actions and the policy space is simply um intractable it becomes uh too huge for you to evaluate the expected free energy for all such policies and in a stochastic

problem setting where um the environment itself is noisy you don't really have a method to choose a subset of this policy space and do classical active inference and this is clearly a computational intractable problem and that's why in literature we always see small grids or small environments when you discuss decision making in active inference but recently uh a new idea was proposed which is called a sophisticated inference and in sophisticated inference it is not really the policy space you are actually um real time trying to think what to do so if you have a belief then you are trying to um evaluate the actions based on that so here we don't have a sequence of policies or things that becomes intractable uh here we are doing basically research in the sense that you are trying to evaluate expected free energy of um this joint distribution of actions and observations and to evaluate the expected free energy at some time uh small T you will also need the expected free energy at the next time step and to evaluate that you will need the expected free energy of time t plus 2 and this basically becomes a research that rolls out in time and uh it's a recursive relation and here it's fundamentally different from the policy space that we saw in the last slide right and is it computationally better in the sense that if you have to plan say 10 times steps ahead comprehensively in classical active inference we saw that the policy space is the cardinality of the action space raised to T so that is the computational bottleneck if you have to uh consider all possibilities but in sophisticated inference it's even worse because uh if you are considering a combination of state and actions so it is computationally worse in fact but a solution was proposed in the sophisticated inference paper that we can do pruning of This research that we can avoid some states and actions uh when you do this research and that becomes computationally very tractable so let's see how pruning Works in sophisticated inference so this grid was discussed in the original sophisticated inference paper and say for this grid given that you have this kind of a prior preference distribution so this white square which is the goal state the most preferred State and you have a uniformly decreasing preference for states that is far away from that goal state then basically if You observe yourself at some observation at time T then basically what you're doing is you're considering the consequence of available actions from that observations and basically you can use the projection of your belief and set a threshold for those beliefs to kind of maybe ignore some actions and ignore some observations and what you will find is that it it's no longer at research it's a subset of research and it's computationally way uh efficient than doing the whole research right so here you have avoided a lot of combinations and this becomes a computationally tractable problem this is because you have decided to avoid some actions and um observation so you say essentially

compromising on those possibilities which might give you a higher reward or which might be more optimal than the consequence of this partial research but this works so this simulation was presented in the paper and it works the agent uh kind of learns to do uh what needs to be done if it plans in this forward direction of planning uh in a pruned way but and basically you can computationally show that even for a small search threshold um you can drastically decrease the computational complexity so if you decide to do the whole research um with the planning depth uh the computational time is exponential and quickly you can't do much about it then but if you decide at such threshold which is even very small you can see that this problem becomes computationally tractable and this demo uh on sophisticated instances are available in my version of my mdp and it's going to be integrated to the original by mdp soon um so but the key point is that the pruning of that research requires us a well-informed prior preference like this in the sense that here the agent is aware of how desirable our neighboring states it not only knows about the final goal set it also knows about its neighboring state and given such a prior preference we can see that for a planning depth of three the agent is getting stuck basically in this local Maxima of Prior preference but with sufficient planning depth it is able to overcome this barrier and reach the goal State uh in this great problem and the question is that what if the agent only knows this final state it has no other knowledge about what to do and in this case what does the agent do so this is the problem that we are trying to address and in the absence of a meaningful prior preference like this the agent has basically no way of reaching the goal State um other than by random exploration it has no way of planning because it cannot do uh planning eight time steps ahead because it's computationally intractable uh if it chooses to do the full research right and so if so for a grid like this what do you do if you get a sparse prior preference which is not well informed and as highlighted in the previous slide your research is now blind uh you have no way of pruning that research and you have to do the full research in in this scenario so as you might have thought um there are two solutions to this either you have to find out a way to do the full depth planning uh or you have to learn a meaningful Prior preference which will enable you to do uh this pruned research so we are going to discuss these two solutions uh in this presentation uh for a given scenario like this so the first solution uh to do full research is basically using dynamic programming and dynamic programming is a well-known idea in operation research and industrial engineering and many in many branches of engineering and the basic idea is that you solve the subparts of a bigger problem first and then later try to integrate the solutions of these sub problems uh to do uh optimal decision making so in this

scenario imagine that you're trying to plan for the Last Action so earlier we started from the first action we started from the present and try to predict what is happening in the future in in so your direction of planning was basically forward in time but imagine that you are trying to plan only for the last time step where you are just near the goal State and right going to that good state right so you are trying to make a decision for that last time step which is capital T minus 1 and your projections uh to the next time step or the last goal state will can be done because you have access to this model of the world uh using this transition Dynamics or the B Matrix in active inference so for this single time step which is a sub problem uh you are actually evaluating a table of expected free energy that tells you if you are in say this observation what is to be done which is for the last time step and basically uh you can do this in state space or observation space so this can be done uh using the A matrix and D Matrix together where you can do planning either way right so the question is that if you know what to do in the last time steps that you might be thinking how do I know that I'm in the last time step it is all about imagining you have sorry just last thing we heard for a second was it's all about imagining yeah so just pick up from there it's all about imagining okay so if there was a connection issue yeah just for a few seconds it's all good then now oh sorry about that but so um so what I'm trying to say is that you are trying to imagine what you will do if you are in time capital T minus one which is the last time step for your planning Horizon and if you are in that time step what do I do so this table represents all such scenarios that if I am say at observation 3 at time T minus one what do I do and this quantity here I'm only considering the risk term or the purposeful term uh and this term represents that policy and what if I do this um backwards till time T minus one so if I'm if I know what to do at time uh capital T minus 1 then this table can inform what to do at capital T minus 2. so rather than planning forward in time what I'm doing is basically stacking many tables together just by fixing a capital T of planning and given that I have all such stack tables then basically what I can do is use them to take decisions forward in time and what we have observed is that this idea works that um I can calculate the expected free energy backwards uh step by step considering them as sub problems and the fundamental difference is that in sophisticated inference uh to calculate the expected free energy at time small T you don't know what is the expected free energy at time team plus one so this becomes a three search so that you have to calculate this first and to calculate for t plus 1 you need t plus 2 and so on but here because you are calculating it backwards in time you already know what is uh the expected free energy for p plus 1 and you base it's basically the same equation it's just that you're doing it backwards in time and

pictorially in sophisticated inference you are trying to do a research but in the dynamic programming algorithm you're doing your planning backwards uh using tables and given your planning Horizon is sufficient enough for a problem what we have seen is that the agent will be able to take optimal actions forward in time so in the paper we are also proposing uh one algorithm for sequential form DPS using uh this backward planning in time and we were able to scale up simulations for grid spaces which was previously intractable without neural networks um so that was the first solution so the second solution is that um so in the first solution it was fixed that you only get this past private reference you don't get any other information you only get your information about the final goal State and all you get is the model of the environment which you learned and you basically have to take decisions but the second solution of course is that if you are allowed you can attempt to learn a prior preference which is Meaningful like the one we saw in the previous slides which has the information about the other states also but how do you learn it right so there is a seminal work from optimal control literature that talks about efficient computation of optimal actions and in that work there is a quantity similar to our prior preference which is called the desirability function so for example in this grid World um here darker colors are more preferred so if this crosses your final gold state what the agent in this paper is trying to do uh or the said learning method in this paper is trying to do is learn this desirability function as optimally as possible and what has been shown in this paper is that if you try to learn the desirability function using a particular learning rule it is computationally far more efficient than even Q learning which is a well-known uh reinforcement learning algorithm so Q learning is a well-known computationally optimal algorithm but in this paper that particular learning rule for learning this kind of a desirability function is way faster and this approximate represents how different it is from the optimal desirability function so this desirability function is nothing but our prior preference and as mentioned learning said is orders of magnitude efficient than learning the Q function in Q learning so this is the particular learning rule depending upon the reward that it gets from the environment and in this particular grid World environment we are only giving it reward at the last step which is similar to the sparse reward preference the agent basically gets no reward uh until it reaches that final goal State and with this learning rule which has a parameter η that controls how fast or slow this Learning Happens so basically we try to study the effect of this learning parameter η but what we observed is that it's very robust it can learn the prior preference reliably even with variable values of this uh learning parameter and what we see is that the agent is able to learn a meaningful prior performance

over time very fast and using such a meaningful Prime preference then the agent don't have to plan a lot it can manage optimal Behavior or purposeful Behavior with very low time Horizons of planning and given these two solutions we could scale up active inference algorithm for decision making and talking about the computational efficiency uh we saw that the dynamic programming method could plant 30 time steps into the future uh with only say a computational complexity of thousand when compared to say 10^{68} for sophisticated inference and the second method which learns the prior preference uh which we call Active inference and it only needs to plan one time step ahead so it is way more computationally efficient in that sense but it has to learn so in the DPFE method we are not learning we are using the spots by preference and doing the whole depth of planning and in the other method we are letting the agent learn this by preference but then we can save planning a lot because it knows a lot of what to do in time right so graphically we can see that um the DPFE method is really computationally efficient and when plotted Against Time uh in the AI 50 equal to one method um it is basically computationally cheaper and yeah so it scales uh very fascinatingly well uh with Higher and Higher and complexity of the environment so we tested it uh this these methods in very huge State spaces like say 900 States compared to um State spaces which has dimension of say 5 or 10 in the usually seen active influence literature um so I want to kind of emphasize that we are not using any neural networks here we are using um explainable active inference agents are doing all necessary matrix multiplication so that we have access explainability to every computation that is taking place in this in these algorithms and when tested on these grids first we validated this on a smaller grid with understates and we observed is that when compared to Benchmark reinforcement learning algorithms like queue learning and dyna dynacy is a model based reinforcement learning algorithm and we compared our newly proposed agents which is DPFE and aif and we saw really good performance and aif agent is slightly worse and that's because it's only planning one time step ahead right but the DPFE agent who does full time planning it's performing as good as benchmarking personal learning algorithms and we tested it with um bigger and weaker grids and when we introduced stochasticity in the gold state in the sense that when the agent had to navigate to stochastic gold state so we changed the gold States at every 10 episodes and what we observed is that the Dyna queue took more time than our bpfe agent to kind of recover and do well so DPFE agent in this stochastic environment performed really well even better than the dynacy agents and you could also see that the AI phase and is recovering um faster but not as good as the other agents so basically that was the result in the

paper and the methods So yeah thank you for listening and I'm open for discussions all right awesome wow very cool okay well just as we start the discussion um if anyone wants to add anything first just how did you come to work on this project uh were you studying active inference and you came to this question as being interesting or were you working on the planning and came to active inference as a method um yeah so little bit background about myself so I studied physics both in my undergrad and post that and towards the end of my post graduation I got interested in things like Game Theory and reinforcement learning and I joined uh for a joint PhD uh with Professor Adil and Professor Manoj and so Professor Manoj is a control theory person and a deal is a neuroscientist and in the beginning of my PhD I started reading active inference literature and I wanted to implement that in problems and I was always fascinated with explainable active inference uh about this idea of expected free energy trying to minimize risk as well as expected ambiguity it's also not just making the agents work but also being able to tell how they work is what fascinated me in active inference and initially I try to implement them and there is a conference paper which we published in which we compare it for a similar gridworld task and that then I faced this problem of scaling active inference and then I started working on sophisticated inference um yeah so basically these methods came out of the need that I wanted to scale the map up and I always had been skeptical in using um deep active inference because um I didn't want to use neural networks to do planning Because deep reinforcement learning in itself is a huge field and um if you if you are about just scaling active inference then maybe just do deep reinforcement learning and that's what I thought yeah so that's basically the background and yeah that's how it came about all right I'll go first to a question in the live chat and we'll just probably discuss various aspects because there was a lot in your presentation so NL Dawn writes I wonder when it comes to Computing the expected free energy over a Time Horizon what kind of mean field approximation was used to factorize Q of s store so so I assume that the question is about [Music] the expected free energy in DPFE and to calculate this basically so in my simulations I use belief propagation for this belief queue and you can also also use variational message passing or marginal message passing that's not a problem but so once you have this belief queue um what do you do right so yeah so when I say you imagine Q for a time then what I use mostly is one hot Vector so to take decisions you use the belief that you get out of your perception step in active inference but for imagining these are hard tables where they are one hot Vector so given say you have in your generative model has 10 states uh your the queues that you use are precise cues for planning but for decision making you use the impress

imprecise mean field approximation queue that you get from the perception step so I don't know if that answers the question um but maybe I I also want to think I also need to think about more about the approximations that may be there in the cool yeah they can they can write more if they want um let's talk a little bit more generally about preference learning so in the context of the active inference generative model we have a mediating between observations in Hidden States and learning makes a lot of sense it's about learning the mapping between observations in Hidden states of the world and then we have learning the consequences of action and how things change Through Time and preference learning is learning on c and you've highlighted that that this is a very um you know interesting variable to learn and I'm curious how does how do we learn to learn the right thing how do we know that we're learning an Adaptive preference and then how does that preference learning reduce cognitive overhead or computational complexity yeah great if you are trying to learn a preference then that assumes that there is something to Chase or there is something to maximize like a reward so in this say uh in a reinforcement learning setting there is clear reward that's coming from the environment that you're trying to maximize and say for this grid you get that reward only in the last step that which is that that's that's what makes this problem hard so if you are getting rewards at every time step then basically you know what to do that I just have to pursue that reward at every time step right but here you might have to take say 15 time steps ahead to get that one reward and that basically is hard to do so here um if you are trying because I'm trying to learn a prior preference um there should be a reward structure that uh that exists in the environment if the environment doesn't care about what I do or if I can't Define what is good or bad then there is no meaning in learning the prior preference right so here um the reward is what controls the learning of the Sprite difference and the good thing is that even if only I get the reward at the last time step um I have my B matrices and I have experience of transitioning from different states um which I reached this final gold state this algorithm that we are using or the learning rule that we are using is precisely learning a similar thing in optimal control that paper I introduced which is called the desirability function and given that you have this desirability function so imagine that maybe I am starting from this state then I just have to look at my nearest neighbors to take a decision if the stated um State below the state is more preferred than I only have to plan one time instead ahead and that's the optimal decision I have to take so learning this Preference reduces the cognitive load in the sense that I'm only now looking at nearest neighbors I don't have to plan all the way up to this final good state and I will learn this as efficiently as possible

because one it was it is a guaranteed algorithm we tested its robustness and it is also informed by the reward that I get from the environment and if yeah so if there is some way uh to kind of Define what is preferred uh then you are guaranteed to learn a private friends that's meaningful by this algorithm so if only nearest Neighbors and one time step inference is being used then what prevents this kind of preference learning agent from being stuck in a local Optima um yeah so there won't be local Optimas here that's the point like um why would there be a local Optima uh if I am learning it using rewards uh yeah so if if there is local Optima then it will be getting stuck in the local Optima but there won't be one uh if you're learning is this way because you're learning it from your own experience and how you got the reward at time uh sometime and what we have observed is that uh it's very gradual and there is no glitches or local Optimas when you learn such as so it's kind of back filling a so that there can be a smooth path yeah to the distal goal yes so it in the animation kind of makes it look like a back feeling but um you're actually learning it from your experience forward in time right so I observed a reward when I transitioned from this state to the state so this must be good uh then the in the in the next time step I say okay this date is responsible for taking me there so this might be good also but not as good as the other one so it looks like it's learning backwards but it's actually learning from the real um into $t + 1$ plus one experience okay you know so in that way um you can't have local Maximas because yeah this is the learning rule that does that so I don't know how to answer it more systematically yeah yeah um what what real world situations or or what real life situations do you kind of see this kind of preference learning happening in yeah great question so so there is one paper that came out from our lab where neurons are laying learning the game of pong um so there was so if the paper is called dist brain uh and the device is called strain and what they have accomplished is that they manage to culture neurons on a silicon chip and they gave it a good uh feedback signal when it successfully tackled the ball or played the game well and gave it a shock uh when um it made mistakes and what we saw is that in the paper what we see is that it learns to play the game over time right and in in such a scenario imagine that if I encounter say a positive thing in the world then I might associate um what is good with the states I observed in the past and so on right so I learned step by step what is good or bad and that's a reasonable hypothesis to make so if if I want to say get fitter then I might associate going to gym as a good thing then I might also associate walking to the gym as a good thing then I might associate uh say wearing my shoes as a good thing so learning prior preference that way makes sense I would say but yeah this is a computational solution to the problem of curse of dimensionality but testing this on the real

world is definitely uh what we should do and um yeah I also look forward to doing okay let me let me try to run a real world situation by you and see if this if this connects um so we wanna we want to turn in we're rewarded by turning in an essay that gets a high grade and so there's many steps between starting the idea of the essay and seeing that preferred sparse outcome which is the grade um and we we do all kinds of things and then we learn okay well um this one where I did get a good grade it was well formatted and then that kind now you sort of expand the umbrella of your preference out and then what brought me to getting it well formatted well I did it on time and then you kind of work all the way to the ideation phase so that in the future you can do one step inference yes exactly and just take it step by step as a skill instead of needing to do a 15 time step inference over all the possible tree structures so you've kind of used your embodied learnings to simplify the structure of the problem exactly yeah and can we look at the computational complexities um again of the of all them so here CIF stands for the classical active inference and to do the full Horizon of planning um it's around 10 to the power 18. so this is for this particular grid example with 100 States and four available actions so this is for a special case and I just wanted to put numbers to put things in perspective so for this particular example if you are trying to do it with the policy space thing um you will have to do um 10 to the power 18 computations for one step of planning or in one um instance in sophisticated inference it's even worse because uh there you go the state space Also matters uh and that won't work so this third line was supposed to kind of show that even for a time um or rise enough two it's really hard to do sophisticated inference if you have to do the full planning but with dynamic programming when you're planning backwards um you can attempt to do the full um and that's only a thousand computations but with learning preference it's even low uh when you do it just one time Step Ahead wow this is quite quite Stark and in the branching time active inference work in a previous model stream we also saw some computational complexity estimates but I think these are really clear um first thing it made me think of is no one said sophistication was cheap I mean it's it's astronomical how costly it is for um maybe even only at two or three or four or five potentially starting to get up to so it's increasing the the complexity of planning um radically so it's very interesting pedagogically as we think about like as we imagine active inference intelligence architectures but this makes it pretty clear that it's not something that you can just enumerate over and so I think it's very um very creative and important what you've done by connecting this to principled methods of computational complexity reduction rather than potentially effective but ad hoc methods of complexity reduction like neural networks which might work when they work but then once

those start to get bloated now you have no principles and no efficacy exactly yeah and sophisticated inference I must note that has its own benefits when compared to dynamic programming uh in the sense that when you're planning forward you are taking into consideration all the possibilities right but when you're planning backwards say for example it's like cue based exploration um like if you have to go somewhere get a queue and then navigate to the pool State then planning backwards might not work so this is what this was one feedback I got and I discussed uh the work with um some people so um that's a thing to note about dynamic programming but the other place where you learn prior to friends I believe it's not a problem but yeah dynamic programming is computationally cheap but it also come with its own limitations I must note here just to restate that in dynamic programming with a Bellman optimality we're solving backwards and so it's kind of like the Checkmate is what we want and so now we're working backwards to the present but we end up not exploring counterfactual end points we don't work back to the present to end points that we're not interested in exactly so that's why it's so ruthless but on the other hand it's a much more constrained search yes so yeah so what I'm thinking about these days is that we should also consider uh combinations of these two uh where I can afford to kind of ignore such counterfactual things um there I can do dynamic programming uh and maybe I can do as two steps uh of planning so if it's a queue based exploration uh say reaching queue uh is one task and from queue to the reward is the other task so I can separate these two and use dynamic programming for them and that's computationally cheaper but also kind of preserves this idea of um having to do it yeah but these are all future things I am thinking about cool or ask another question from the chat Alex wrote do you have ideas or developments on nested models where different scales could have different one time step ahead and speed of execution so how does this model play out in nested models and how how do we think about this forwards and backwards in time in the speed of execution and nested models okay so I might need a little more context here like um when you say nested models um do you mean hierarchical models yeah yeah so maybe um something like there is a observation I get on a fast pace and I do some inference on that but then I use this inference uh to kind of do or maybe uh and that basically this inference becomes the observation for the next state and that's what nested models mean right yes so I might have to actually think about that in terms of a task and then think about it but honestly I I haven't um thought how this might work in that context but say for a task like uh so in terms of navigation if you think about it you can think about say rooms environment so that's where uh that's one of the examples I can think about

where we can apply this so imagine maybe you have a collection of rooms and as an agent you have to first figure out which room to go and then you have to navigate inside that home and basically you can do uh inference in two stages or decision making in two stages uh and you have you will have to separate your decisions in in those stages inside the room you can do say dynamic programming to navigate your optimal uh path but you will always have to kind of have two stages of um decision making and um maybe different methods work better at different stages but this would be uh better in you know I mean this discussion would be better to better in in a well thought out task I would say I I don't have a answer that might be fitting for everything but yeah we've seen almost exactly that kind of hierarchical simultaneous localization and mapping Slam in a robotics case there have been active inference models on that um yeah would it be possible to sort of do um one of these methods at one level of the nested model and then have another computational method applied to another um or is like if you wanted the advantages of one in one place like can you mix and match with these different methods even within one simulation yeah I I I definitely think that's possible um but yeah we'll have to maybe try so all my methods are um one stage so it's not a hierarchical model and but I I strongly believe that it would also work in an estate model where you can have um two methods of decision making working together in say for a room example I just mentioned oh could you go to the slide on Z learning cool yeah so I noticed um here and in the great paper that you had several total of citations yes and so and this introduction of the Z learning is is a novelty with respect to the active inference field so could you explain a little more what is the Z and what is it that enables such a a rapid Improvement in the Z with respect to the Q yeah okay so uh to give some context um context about this paper it's it talks about a linear method of decision making uh in a particular class of mdp so given that you are having a Markov decision process where your actions can be based on States and not accents so when you think about actions in a say in for example a gridual task you think about left right and um North South right so if I take a north then that has a consequence of uh consequence on state space but in this paper um they're introducing a class of mdp where decision is itself in terms of states so if I am saying State S1 my decision will be uh depending upon the other state so my decision will be based on the other states I want to be in the next time so uh it's a really definition of decision making uh in terms of the state space rather than decisions being something else like left right and down up so given that such an mdp exists where I can take decisions in terms of states they have showed that computationally you can take uh decisions in linear complexity however big the problem is so for that to so to enable decision making in

terms of States you should have a sense of what is good and bad for the states so in this gridward example uh you have a desirability function which is C so C is the desirability function which talks about how desirable is a state and if I have a c function then what they have showed is that I can take decisions with linear computational complexity for this particular class of mdp so if my mdp allows me to take decisions in terms of States it's linearly uh it's only linear complexity for that thing so this graph is basically comparing how you can learn desirability uh better or faster right so Q learning if you are familiar with Q learning it's basically a table based method uh where you have um desirability of actions given a state so given a state you know what to do that's basically the queue Matrix but in terms of C uh or the C learning method it's only about States uh you're only learning how desirable a state is there is no concept of actions and this is exactly what our prior preference uh is in active inference where it is a distribution that quantifies how desirable or non-desirable states are right so given that they have shown that you have you can learn the C Matrix faster and that's optimal and it's way faster than even Q learning then I thought okay why not try to learn C the same way as C is learned in this paper and using this say so there is a similar learning rule for learning C which is called set learning in that paper and when I attempted to learn see what I've seen is that it learns really fast a useful Preference that lets me take decisions or let the active inference agent take decisions um only by say one time step of planning and yeah so basically that took the story so this idea of C being easily learnable is in that paper okay let me try to um restate that since I think it's a very interesting augmentation of active inference so we're going to be learning C for all the reasons we discussed earlier we're going to learn C analogously to how todomov presented Z learning and in the Z learning instead of learning um for example updated posterior probabilities on actions and then using actions to navigate amongst states that emit observations yeah we're going to kind of bake the action into the states so that really we're learning transitions among states directly yeah and and to connect this to the free energy principle and and how it is playing out with with active inference C is not just our desirability function that's one way of thinking about it that's why we call it preference but also C is our expectation and so that is what allows us to on one hand use the language familiar to reward and preference learning like the agent is ending up where it likes to be but also the expectation based um definition of c these are the same thing allows us to talk about that as a path of least action or as the most likely outcome or the least surprising outcome and because we've defined it what we want as the least surprising outcome then we can use variational free energy to bounce surprise whereas you can't use simply a variational

method to bound or even necessarily approximate reward itself but if you say I prefer what I expect and what I expect reduces my surprise and I'm going to balance surprise then you get both that sort of Behavioral reward seeking couched in a surprise bounding surprise minimizing physics framework yeah yeah that's a beautiful way of saying it yeah thank you well what are your um next uh exciting steps or directions or what what ways do you want to take this work yeah great um if you're talking about only this work um what I want to do next is uh think about cue based exploration tasks I first of all address the limitations of dynamic programming so it's if you have say a queue to explore in this grid first and that's more optimal I want to see how the uh expected ambiguity term in the expected free energy is useful and should be made use in the dynamic programming strictly in that sense um but more generally I'm also looking at other ways of making decisions in active inference like there are works from Professor a summer from CBS reckon he talks about how it's How neural networks are doing active inference and basically decision making there is um really uh more efficient in the sense that it's like queue learning so in there he's making clever use of the variation free energy to learn good State action mappings and it's it's a very drastic um change to what we are used to in terms of expected free energy so there is no concept of expected free energy uh in in that work it's all about learning what is good and bad directly from variation for energy so I I find that also fascinating I want to kind of explore that more and see how decision making uh in this way is better or worse or should be thought about should we even reconsider ways of decision making because active inference only talks about variational free energy and that's the central tenet everything else is your interpretation of that right so yeah that's another Direction I want to work interesting way to say it um definitely the variational free energy which is a functional of our variational um distribution q and the data y the variational free energy is kind of like the real time homeostasis like yeah how are things making sense given what I believe and the incoming data and then to extend that kind of a sense-making framework into decision making we've seen a lot of different methods expected free energy is um a common one but for example there's been free energy of the expected future as eef and there's other constructions that have different methods um and then you point it also to Professor samura's work with um the sort of uh relationship between the variational free energy on the base graph and the loss function in a neural network and all those relationships that's very exciting work too um I guess kind of in closing just as a last question or thought you're coming close to the end of your PhD so just in the time that you've been a PhD student how have you seen active inference develop or what what feels different to you today near the

end than when you were yeah fresh-eyed and excited several years ago yeah that's that's a really great question and uh I am also very excited of how the field has evolved and frankly I started from this reinforcement learning background and the physics background and when I started reading it was only say one or two papers and I did not quite understand much of it uh it was only when I started uh implementing it using calls um Matlab scripts I kind of understood okay this makes sense and I like it and but within uh say one or two years I saw a lot of papers coming in uh from different directions and people also starting to use neural networks and all this scaling up came and I I at some point I also questioned the need of active inference uh because if you have deep reinforcement learning can which can do many things then why deep active inference and that's the reason why I did not get into that but I still find it fascinating I want to kind of understand deep active inference more than what I know now but I've seen the field growing like anything in in two or three years and many cohorts of people starting to work and in no time it was a seriously taken field other than a field with say two papers with nobody actually knowing what it is so it's really exciting yeah so I really look forward how the field evolves in time and also what I can do after my PhD and so on cool forward in time backward in time what we prefer what we expect yes any other comments or anything else you want to add yeah so please let me know what you think about the paper and what you think about these ideas feel free to let me know I I really look forward to the feedback and yeah thanks so much for this opportunity Daniel and thank you for your time it was amazing I hope people um check out the paper and and get in touch and replicate the code and and take it their own directions thank you see you next time see you next time thank you so much bye have a good day bye